

Exploration

CS 185/285

Instructor: Sergey Levine
UC Berkeley



Part 1:

The exploration problem

What's the problem?

this is easy (mostly)

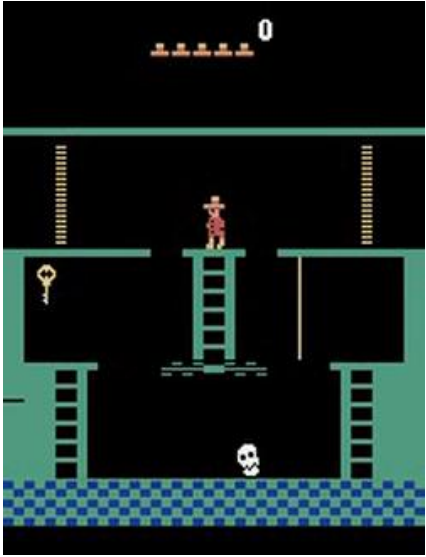


this is impossible



Why?

Montezuma's revenge



- Getting key = reward
- Opening door = reward
- Getting killed by skull = nothing (is it good? bad?)
- Finishing the game only weakly correlates with rewarding events
- We know what to do because we **understand** what these sprites mean!

Put yourself in the algorithm's shoes



Mao

- “the only rule you may be told is this one”
 - Incur a penalty when you break a rule
 - Can only discover rules through trial and error
 - Rules don’t always make sense to you
-
- Temporally extended tasks like Montezuma’s revenge become increasingly difficult based on
 - How extended the task is
 - How little you know about the rules
 - Imagine if your goal in life was to win 50 games of Mao...
 - (and you didn’t know this in advance)

Another example



Learned Policies

Exploration and exploitation

- Two potential definitions of exploration problem
 - How can an agent discover high-reward strategies that require a temporally extended sequence of complex behaviors that, individually, are not rewarding?
 - How can an agent decide whether to attempt new behaviors (to discover ones with higher reward) or continue to do the best thing it knows so far?
- Actually the same problem:
 - Exploitation: doing what you *know* will yield highest reward
 - Exploration: doing things you haven't done before, in the hopes of getting even higher reward

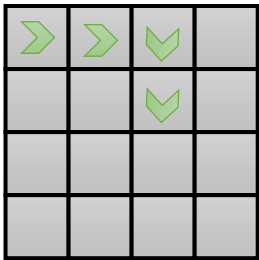
Exploration and exploitation examples

- Restaurant selection
 - **Exploitation**: go to your favorite restaurant
 - **Exploration**: try a new restaurant
- Online ad placement
 - **Exploitation**: show the most successful advertisement
 - **Exploration**: show a different random advertisement
- Oil drilling
 - **Exploitation**: drill at the best known location
 - **Exploration**: drill at a new location

What makes an exploration problem tractable?

multi-arm bandits
contextual bandits

can formalize exploration
as POMDP identification
policy learning is trivial
even with POMDP



small, tabular MDPs

can frame as Bayesian model
identification, reason explicitly
about value of information



large or infinite MDPs

optimal methods don't work
...but can take inspiration from
optimal methods in smaller settings
use hacks in practice

Bandits

What's a bandit anyway?



the drosophila of exploration problems



$$\mathcal{A} = \{\text{pull arm}\}$$

$$r(\text{pull arm}) = ?$$



$$\mathcal{A} = \{\text{pull}_1, \text{pull}_2, \dots, \text{pull}_n\}$$

$$r(a_n) = ?$$

$$\text{assume } r(a_n) \sim \underline{p(r|a_n)}$$

unknown *per-action* reward distribution!

How can we define the bandit?

assume $r(a_i) \sim p_{\theta_i}(r_i)$

e.g., $p(r_i = 1) = \theta_i$ and $p(r_i = 0) = 1 - \theta_i$

$\theta_i \sim p(\theta)$, but otherwise unknown

this defines a POMDP with $\mathbf{s} = [\theta_1, \dots, \theta_n]$

belief state is $\hat{p}(\theta_1, \dots, \theta_n)$

- solving the POMDP yields the optimal exploration strategy
- but that's overkill: belief state is huge!
- we can do very well with much simpler strategies

how do we measure goodness of exploration algorithm?

regret: difference from optimal policy at time step H :
$$\text{Reg}(H) = HE[r(a^*)] - \sum_{t=1}^H r(a_t)$$

expected reward of best action
(the best we can hope for in expectation)

actual reward of action
actually taken

Optimistic exploration with bandits

keep track of average reward $\hat{\mu}_a$ for each action a

exploitation: pick $a = \arg \max \hat{\mu}_a$

optimistic estimate: $a = \arg \max \hat{\mu}_a + \underbrace{C\sigma_a}_{\text{some sort of variance estimate}}$

intuition: try each arm until you are *sure* it's not great

example (Auer et al. Finite-time analysis of the multiarmed bandit problem):

$$a = \arg \max \hat{\mu}_a + \sqrt{\frac{2 \ln H}{N(a)}} \quad \leftarrow \begin{array}{l} \text{number of times we} \\ \text{picked this action} \end{array}$$

$\text{Reg}(H)$ is $O(\log H)$, provably as good as any algorithm

Part 2:

Optimistic exploration in deep RL

Optimistic exploration in RL

UCB:
$$a = \arg \max \hat{\mu}_a + \underbrace{\sqrt{\frac{2 \ln H}{N(a)}}}_{\text{“exploration bonus”}}$$

lots of functions work, so long as they decrease with $N(a)$

can we use this idea with MDPs?

count-based exploration: use $N(\mathbf{s}, \mathbf{a})$ or $N(\mathbf{s})$ to add *exploration bonus*

use $r^+(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \mathcal{B}(N(\mathbf{s}))$



bonus that decreases with $N(\mathbf{s})$

use $r^+(\mathbf{s}, \mathbf{a})$ instead of $r(\mathbf{s}, \mathbf{a})$ with any model-free algorithm

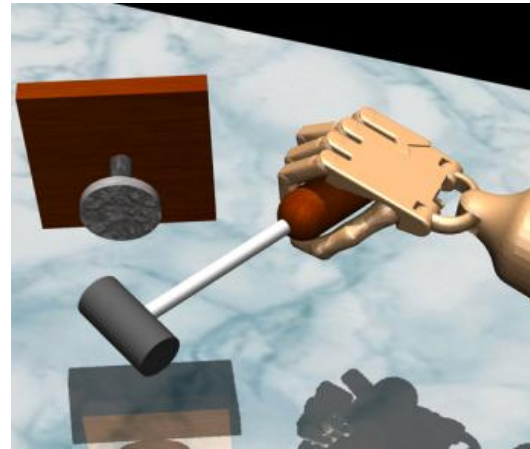
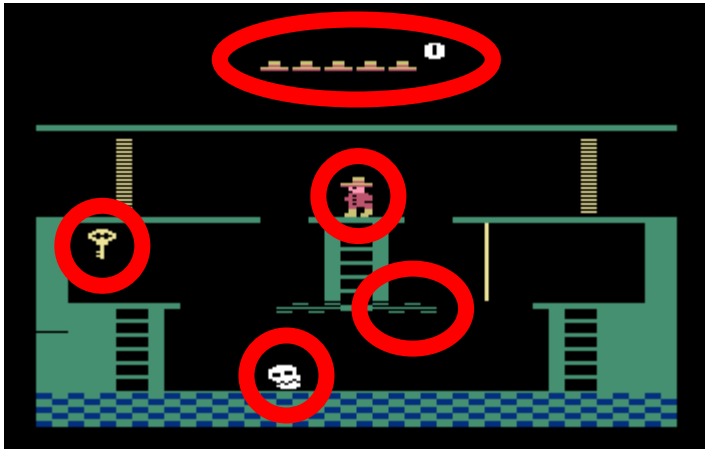
+ simple addition to any RL algorithm

- need to tune bonus weight

The trouble with counts

use $r^+(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \mathcal{B}(N(\mathbf{s}))$

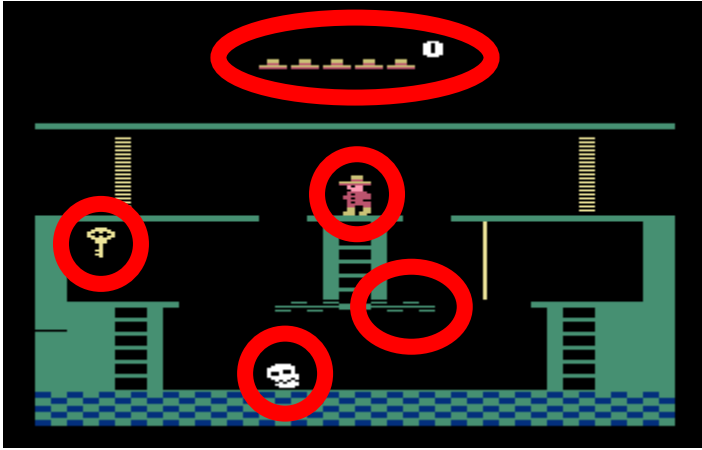
But wait... what's a count?



Uh oh... we never see the same thing twice!

But some states are more similar than others

Fitting generative models

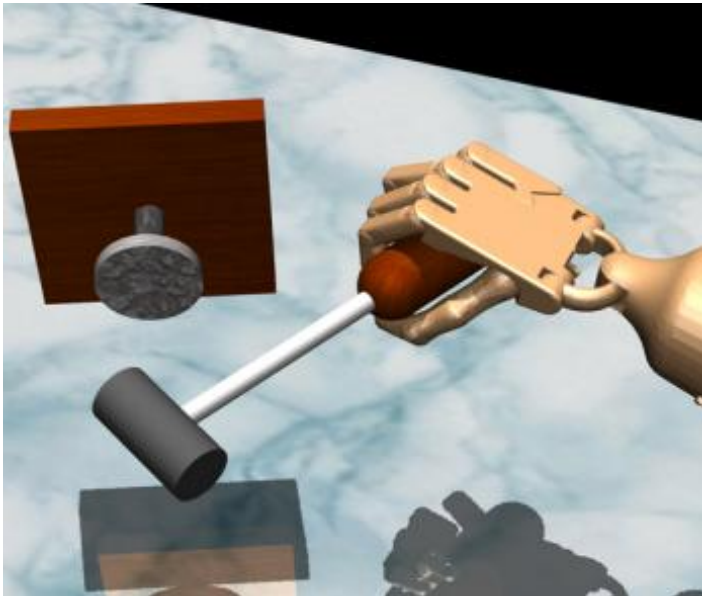


idea: fit a density model $p_{\theta}(\mathbf{s})$ (or $p_{\theta}(\mathbf{s}, \mathbf{a})$)

$p_{\theta}(\mathbf{s})$ might be high even for a new $\mathbf{s}$

if $\mathbf{s}$ is similar to previously seen states

can we use $p_{\theta}(\mathbf{s})$ to get a “pseudo-count”?



if we have small MDPs

the true probability is:

$$P(\mathbf{s}) = \frac{N(\mathbf{s})}{n}$$

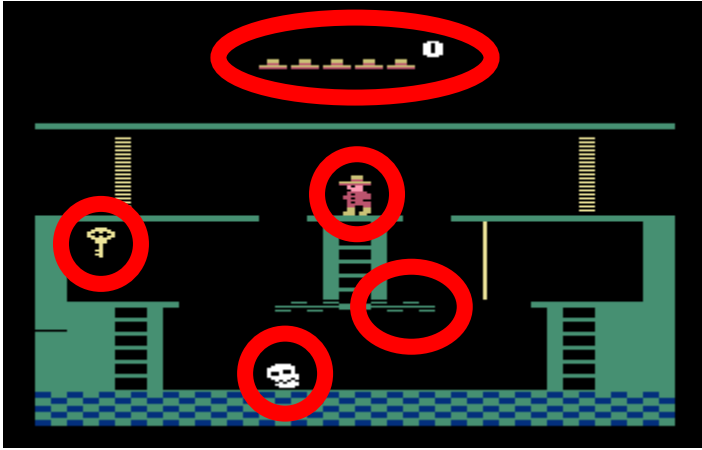
probability/density $\nwarrow$ $\nearrow$ count $\nwarrow$ total states visited $\nearrow$

after we see $\mathbf{s}$, we have:

$$P'(\mathbf{s}) = \frac{N(\mathbf{s}) + 1}{n + 1}$$

can we get $p_{\theta}(\mathbf{s})$ and $p_{\theta'}(\mathbf{s})$ to obey these equations?

Exploring with pseudo-counts



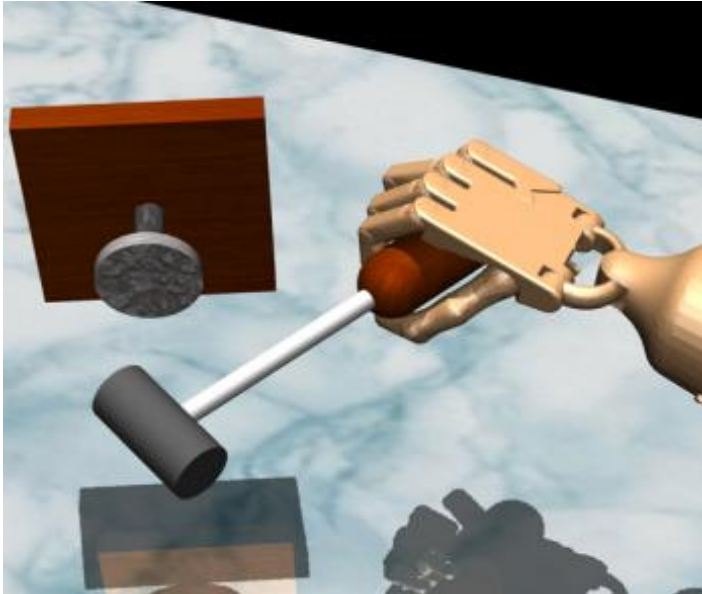
fit model $p_\theta(\mathbf{s})$ to all states $\mathcal{D}$ seen so far

take a step i and observe $\mathbf{s}_i$

fit new model $p_{\theta'}(\mathbf{s})$ to $\mathcal{D} \cup \mathbf{s}_i$

use $p_\theta(\mathbf{s}_i)$ and $p_{\theta'}(\mathbf{s}_i)$ to estimate $\hat{N}(\mathbf{s})$

set $r_i^+ = r_i + \mathcal{B}(\hat{N}(\mathbf{s}))$ ← “pseudo-count”



how to get $\hat{N}(\mathbf{s})$? use the equations

$$p_\theta(\mathbf{s}_i) = \frac{\hat{N}(\mathbf{s}_i)}{\hat{n}}$$

$$p_{\theta'}(\mathbf{s}_i) = \frac{\hat{N}(\mathbf{s}_i) + 1}{\hat{n} + 1}$$

two equations and two unknowns!

$$\hat{N}(\mathbf{s}_i) = \hat{n} p_\theta(\mathbf{s}_i) \quad \hat{n} = \frac{1 - p_{\theta'}(\mathbf{s}_i)}{p_{\theta'}(\mathbf{s}_i) - p_\theta(\mathbf{s}_i)} p_\theta(\mathbf{s}_i)$$

What kind of bonus to use?

Lots of functions in the literature, inspired by optimal methods for bandits or small MDPs

UCB:

$$\mathcal{B}(N(\mathbf{s})) = \sqrt{\frac{2 \ln n}{N(\mathbf{s})}}$$

MBIE-EB (Strehl & Littman, 2008):

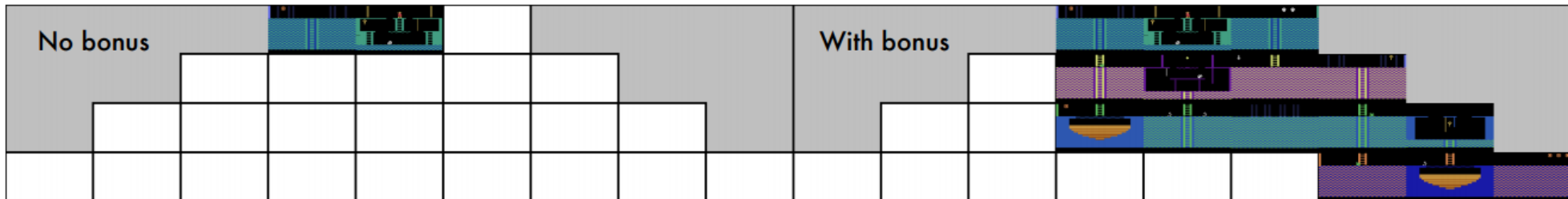
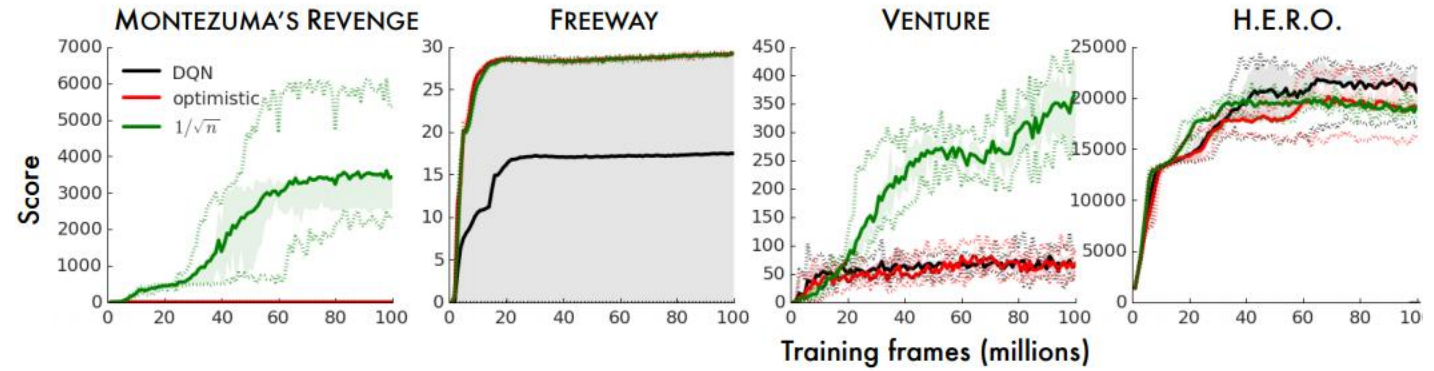
$$\mathcal{B}(N(\mathbf{s})) = \sqrt{\frac{1}{N(\mathbf{s})}}$$

BEB (Kolter & Ng, 2009):

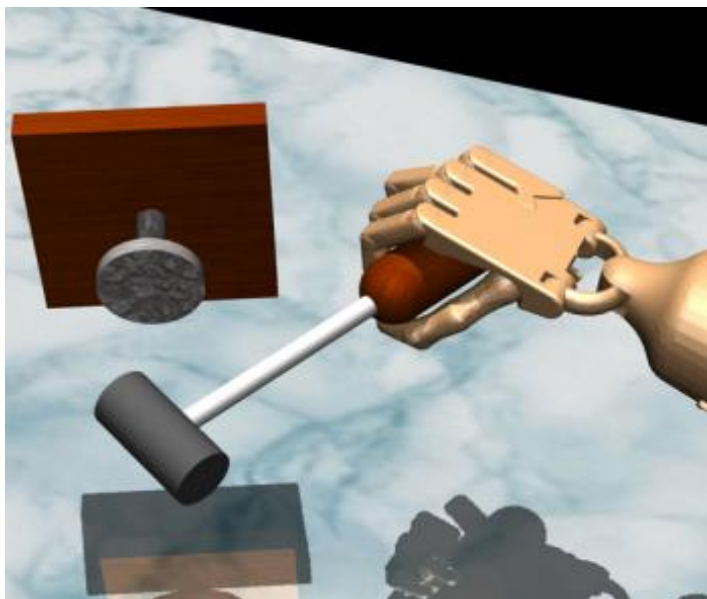
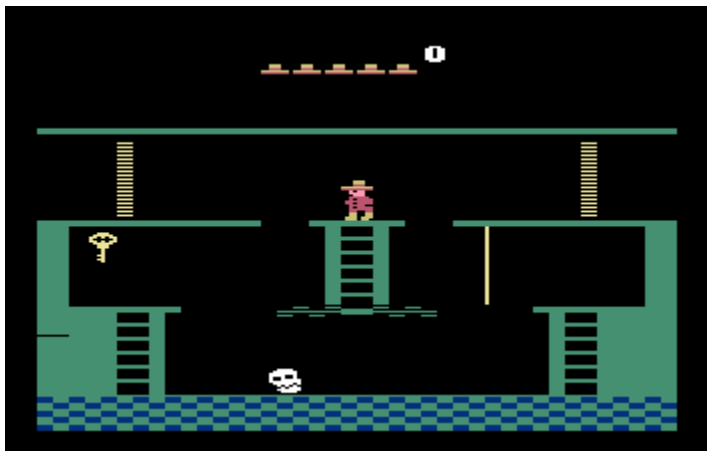
$$\mathcal{B}(N(\mathbf{s})) = \frac{1}{N(\mathbf{s})}$$

← this is the one used by Bellemare et al. '16

Does it work?



What kind of model to use?

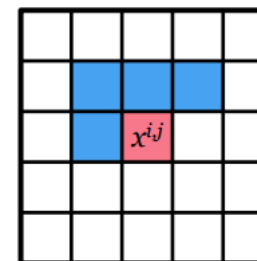


$$p_{\theta}(\mathbf{s})$$

need to be able to output densities, but doesn't necessarily need to produce great samples

opposite considerations from many popular generative models in the literature (e.g., GANs)

Bellemare et al.: "CTS" model:
condition each pixel on its top-left neighborhood



Other models: stochastic neural networks, compression length, EX2

Heuristic estimation of counts via errors

$p_\theta(\mathbf{s})$

need to be able to output densities, but doesn't necessarily need to produce great samples

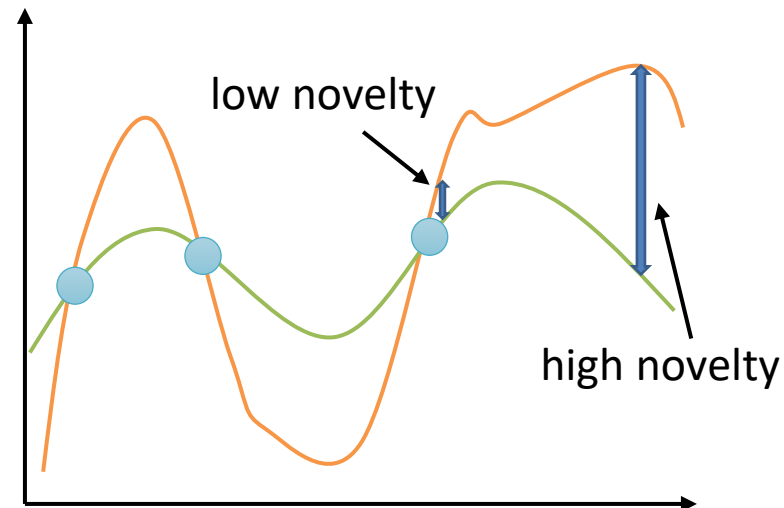
...and doesn't even need to output great densities

...just need to tell if state is **novel** or not!

let's say we have some **target** function $f^*(\mathbf{s}, \mathbf{a})$

given our buffer $\mathcal{D} = \{(\mathbf{s}_i, \mathbf{a}_i)\}$, fit $\hat{f}_\theta(\mathbf{s}, \mathbf{a})$

use $\mathcal{E}(\mathbf{s}, \mathbf{a}) = \|\hat{f}_\theta(\mathbf{s}, \mathbf{a}) - f^*(\mathbf{s}, \mathbf{a})\|^2$ as bonus



Heuristic estimation of counts via errors

let's say we have some **target** function $f^*(\mathbf{s}, \mathbf{a})$

given our buffer $\mathcal{D} = \{(\mathbf{s}_i, \mathbf{a}_i)\}$, fit $\hat{f}_\theta(\mathbf{s}, \mathbf{a})$

use $\mathcal{E}(\mathbf{s}, \mathbf{a}) = \|\hat{f}_\theta(\mathbf{s}, \mathbf{a}) - f^*(\mathbf{s}, \mathbf{a})\|^2$ as bonus

what should we use for $f^*(\mathbf{s}, \mathbf{a})$?

one common choice: set $f^*(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$ – i.e., next state prediction

even simpler: $f^*(\mathbf{s}, \mathbf{a}) = f_\phi(\mathbf{s}, \mathbf{a})$, where ϕ is a *random* parameter vector

